

0-Kernel, 0-Copy Data Movement: Scaling LLM Point-to-Point and Collective Communication from Standard Production to Hyperscale

Author: Ping Long, Chief Systems Architect, Lead Researcher, [SiliconLanguage Foundry](#)

Contact: [LinkedIn](#) | [GitHub](#) | [Okernel.ai](#) | plongpingl@gmail.com | fellow@0kernel.ai

1. Executive Summary

The architecture of AI infrastructure data planes must dynamically adapt across the Large Language Model (LLM) scaling spectrum. This paper provides a unified architectural blueprint that spans the entire scaling continuum, defining exactly when to use synchronized collectives and when to shift to point-to-point asynchronous transports based on cluster size. While traditional Bulk-Synchronous Parallelism (BSP) and collective communication libraries such as the NVIDIA Collective Communication Library (NCCL) are optimal for synchronous environments at lower scales, transitioning to the 100,000+ GPU scale fundamentally breaks these legacy paradigms. At hyperscale, compute infrastructure inevitably spans multiple datacenter buildings, disparate AI zones, and heterogeneous network fabrics, precipitating catastrophic performance degradation under rigid synchronization.

The contemporary paradigm shift is defined by the transition from rigid, synchronized collective barriers to asynchronous, heterogeneous, and disaggregated execution across cloud boundaries. Workloads such as disaggregated inference—which separates compute-bound prefill phases from memory-bound autoregressive decode phases—and sparse Mixture-of-Experts (MoE) routing require fine-grained, dynamic point-to-point asynchronous streaming. Point-to-point libraries like NIXL mandate hardware-enforced ordered delivery mechanisms, rendering them fatally incompatible with modern, multi-path cloud transports such as the AWS Elastic Fabric Adapter (EFA), which relies on the Scalable Reliable Datagram (SRD) protocol to maximize bisection bandwidth via out-of-order packet spraying.

This research report provides an exhaustive architectural analysis of the communication bottleneck across scales. The analysis delineates the precise mechanical failures of standard NCCL and NIXL at hyperscale, evaluates the transition to next-generation host-driven data planes engineered by entities like Meta (NCCLX), and presents [fabric-lib](#) as a highly specialized, zero-lock-in abstraction tailored strictly for custom hyperscale inference and sparse P2P workloads. Finally, the report provides production-grade, step-by-step implementation blueprints for migrating inference clusters to a 0-kernel, zero-copy, transport-agnostic architecture capable of saturating 400 Gbps links regardless of the underlying silicon ordering constraints.

2. The Scaling Spectrum: Contextualizing NCCL

Before evaluating hyperscale failures, it is critical to establish a clear delineation of scale across the training architecture, as hardware size directly dictates communication primitives.

2.1 <10K GPU Scale: NCCL Dominance

For synchronous training loops under 10,000 GPUs, standard NCCL remains the optimal blueprint. In these localized environments, it effectively maximizes bandwidth and aggressively leverages hardware-offloaded reduction math (such as NVIDIA SHARP) over static, local NVLink and NVSwitch domains.

2.2 100K+ GPU Scale: The Hyperscale Wall

Contrast this with the 100K+ hyperscale wall. Scaling clusters to the 100,000+ GPU threshold fundamentally breaks the assumptions of localized BSP. At this magnitude, the underlying network topology must utilize a multi-tier Clos architecture spanning multiple physical buildings. Under these conditions, physical cross-datacenter latencies cause NCCL's kernel-driven, two-stage FIFO memory copying to fail. To overcome this, advanced architectures like Meta's NCCLX solve the bottleneck by shifting to a host-driven, zero-copy architecture.

2.3 The "Straggler" Synchronization Bottleneck

In traditional BSP workloads, standard collective operations such as All-Reduce and All-Gather impose strict global synchronization barriers across the entire communicator group. The mathematical probability of encountering a "straggler" node—a GPU or host operating slightly below peak efficiency—approaches absolute certainty at the 100,000+ GPU scale.

Because NCCL collectives introduce these global barriers, faster GPUs must wait until every participating process has reached the synchronization point. If a single CPU core is delayed by just 1 millisecond due to operating system scheduling jitter, minor thermal throttling, or localized PCIe bus contention, all other participating GPUs enter an idle busy-wait state. NCCL's reliance on GPU-side spin-waiting wastes critical compute cycles and amplifies microsecond-level local jitter into cluster-wide, millisecond-level pipeline stalls.

2.4 The Physics of Scale: Bandwidth-Delay Product (BDP) Impact

To understand why standard collective libraries fail at the 100K GPU scale, we must analyze the physical relationship between network throughput and signal propagation delay, quantified by the **Bandwidth-Delay Product (BDP)**. BDP represents the amount of "data in flight" required to keep a network link fully saturated.

As the physical footprint of a cluster expands—transitioning from a single rack to multi-tier Clos networks spanning multiple datacenter buildings—the round-trip time (RTT) increases significantly. Because standard NCCL relies on a synchronous, kernel-driven "stop-and-wait" or small-window FIFO staging mechanism, it struggles to keep the "pipe" full as the BDP grows. At hyperscale, the BDP exceeds the capacity of local GPU staging buffers, leading to link underutilization and buffer starvation [15].

2.4 Physical Network Latency Penalties in Multi-Tier Clos Networks

Hyperscale clusters are universally deployed across hierarchical 3-tier or 4-tier Clos architectures to accommodate massive node counts. Standard NCCL relies heavily on a two-stage copy mechanism where clear-to-send (CTS) control messages sit directly on the critical execution path. As communication traverses the upper tiers of a hyperscale Clos network, the latency penalties scale disastrously.

Topology Tier	Switching Layer	Relative Latency Penalty	BDP Impact	NCCL Protocol Efficacy

Intra-Rack	RTSW	1x (Baseline)	Low; easily saturated	High
Inter-Rack / Intra-Zone	CTSW	7x	Moderate	Moderate
Inter-Zone / Intra-DC	ATSW	15x	High	Low
Cross-Datacenter	ATSW Mesh	30x	Severe	Critical Failure

2.5 Cascading Pipeline Stalls and Domino Effects

When NCCL's control-message latency creates a localized communication delay between adjacent pipeline stages, it triggers a devastating domino effect. The delayed communication operations stall subsequent computation operations, triggering cascading pipeline stalls that compound across the entire depth of the pipeline.

2.6 SM and HBM Starvation via Two-Stage FIFO Memory Copying

The most structurally fatal mechanical flaw in standard NCCL at scale is its reliance on GPU-kernel-driven, two-stage FIFO memory copying. This architecture is disastrous for advanced multi-dimensional parallelism strategies that rely on compute-communication overlapping. The device-to-device (D2D) copies consume critical GPU SM cycles and saturate HBM bandwidth.

2.7 The AWS Mitigation: aws-ofi-nccl and the "Libfabric Tax"

While standard NCCL struggles with scale, deploying it on AWS infrastructure introduces another layer of complexity. To enable NCCL over the AWS Elastic Fabric Adapter (EFA), the `aws-ofi-nccl` plugin [27] acts as a critical shim, mapping NCCL's connection-oriented APIs to libfabric's reliable datagram interface.

However, `aws-ofi-nccl` [27] does not fully resolve the underlying architectural bottlenecks, specifically for sparse and point-to-point workloads:

- **Software Reordering Overhead:** NCCL strictly expects in-order delivery. Because EFA utilizes the Scalable Reliable Datagram (SRD) protocol—which intentionally sprays packets out-of-order to maximize multi-path bandwidth—the `aws-ofi-nccl` plugin [27] must perform software-level packet reordering on the CPU. This introduces a "Libfabric tax," adding 5–10 microseconds of jitter to tightly-coupled loops.
- **Static Membership:** `aws-ofi-nccl` [27] remains tethered to NCCL's rigid "MPI World" static membership constraints. It cannot dynamically scale nodes on-the-fly, a necessity for elastic disaggregated inference.
- **P2P Underutilization:** While excellent for dense All-Reduce training workloads, `aws-ofi-nccl` [27] often underutilizes multi-NIC configurations (like the 4x100Gbps setup on p5 instances) when tasked with the sparse, one-sided point-to-point transfers required by Mixture-of-Experts (MoE) and KV-cache disaggregation.

3. Next-Generation Data Planes: NCCLX and Host-Driven Collectives

To shatter the hyperscale communication wall, operators running the world's largest AI clusters have fundamentally rewritten the communication stack. Solutions such as Meta's NCCLX abandon GPU-driven, copy-based collectives entirely in favor of Host-Driven, Zero-Copy architectures designed explicitly to mask latency and eliminate resource contention.

3.1 The Mechanical Shift: Host-Driven, Zero-Copy Architecture

The next-generation data plane operates on a strict 0-kernel philosophy for data movement. By eliminating the intermediate NCCL FIFO staging buffers, systems can utilize the RDMA NIC to issue transfers directly from the user's source memory buffer to the destination memory buffer. This zero-copy paradigm achieves SM-free execution and allows the host to push unsegmented payloads to the NIC, maximizing network utilization.

3.2 Dynamic Queue Pair Load Balancing (DQPLB)

Advanced data planes implement Dynamic Queue Pair Load Balancing (DQPLB) to explicitly manage multi-path load balancing and congestion in software. DQPLB calculates the Bandwidth-Delay Product (BDP) of the specific topology tier and establishes multiple configurable data QPs for payload transmission, dynamically limiting outstanding segments to prevent switch buffer overflow.

4. Disaggregated Inference: NIXL Context and Limitations

While NCCL dominates synchronous training, it fails at inference due to forced bulk synchronization, static rigid communication rings, and extreme CPU dispatch overhead. The dominant paradigm shifts toward highly dynamic, point-to-point asynchronous streaming, driven by disaggregated inference and MoE routing.

4.1 Introduction to NIXL and the UCX Framework

To address the requirement for P2P asynchronous streaming, NVIDIA introduced the Inference Xfer Library (NIXL). Engineered primarily as a high-level abstraction over the Unified Communication X (UCX) framework, NIXL aims to accelerate disaggregated serving, KV cache offloading, and LLM-aware request routing.

4.2 Critique: NIXL's Limitations on Cloud-Native Fabrics (Pre-1.0)

Historically, NIXL exhibited fatal limitations on cloud-native fabrics due to its reliance on the UCX backend and ordered delivery mechanisms. Because hyperscale cloud providers optimize their proprietary networks by intentionally abandoning hardware-level ordering (e.g., AWS EFA using SRD), NIXL's assumption of in-order delivery broke down, causing severe head-of-line blocking in software receive buffers.

4.3 Evaluating Open-Source Alternatives (Mooncake and DeepEP)

The industry saw open-source P2P alternatives emerge, yet they suffered from severe hardware dependencies: Mooncake Transfer Engine is limited by standard ordered RDMA, and DeepEP relies strictly on GPU-initiated RDMA (IBGDA), creating vendor lock-in to NVIDIA ConnectX NICs.

4.4 The NIXL 1.0.0 Evolution

With the release of **NIXL 1.0.0** [28], NVIDIA directly addressed the architectural gaps that previously hindered its adoption on cloud fabrics like AWS EFA. Moving beyond preliminary support, NIXL 1.0.0 [28] introduced deep optimizations for the Scalable Reliable Datagram (SRD) protocol, repositioning it as a production-grade standard for inference scaling.

- **Native EFA Backend:** NIXL 1.0.0 [28] implemented a native Libfabric backend, eliminating the previous reliance on the generic UCX plugin that suffered from extra memory copies. This enables

direct integration with EFA's multi-rail capabilities, seamlessly aggregating 4x100Gbps or 2x200Gbps NICs.

- **Zero-Copy Path and NVLink Shared Memory:** Intra-node transfers now utilize an optimized `shm` provider via NVLink, bypassing the network transport entirely for local GPU-to-GPU KV cache movement.
- **Descriptor Traversal Optimization:** By streamlining `nixlDescList` iteration in the Device API V2, NIXL [28] significantly reduced the hot-path overhead previously associated with many small scatter/gather operations typical of MoE architectures.

While `fabric-lib` remains a leaner, highly specialized tool for custom rust-based engines (ignoring ordering entirely), NIXL 1.0.0's [28] stability, framework integration (vLLM, SGLang, NVIDIA Dynamo), and resolved EFA bottlenecks have solidified it as the standard for production disaggregated inference.

5. Custom Hyperscale Inference: Transport-Agnostic Abstractions and Implementation

While NIXL 1.0.0 serves as the production standard for general disaggregated inference, custom hyperscale engines and sparse P2P workloads demanding absolute maximum throughput require pure 0-kernel abstractions. To break vendor lock-in and fully harness proprietary, out-of-order cloud transports for these specific workloads, infrastructure teams can leverage transport-agnostic abstractions like `fabric-lib`. It matches performance on ConnectX-7 hardware while extending that capability to AWS EFA without the overhead of generalized frameworks.

5.1 Uniting ConnectX and AWS EFA under a Single Abstraction

The fundamental insight driving `fabric-lib` is that both NVIDIA ConnectX and AWS EFA are highly capable of high-throughput, reliable, unordered message delivery. `fabric-lib` abstracts these backend protocols via a unified TransferEngine, transparently managing multiple NICs per GPU to saturate 400 Gbps or 800 Gbps links.

5.2 Host-Proxy Architecture and the IMMCOUNTER Primitive

`fabric-lib` completely bypasses the need for GPU-initiated RDMA (IBGDA) by utilizing a highly optimized CPU host-proxy thread. Because it embraces out-of-order transports, it relies on its novel IMMCOUNTER primitive (using one-sided WRITEIMM operations), which abandons strict packet ordering for order-agnostic completion notification based on fundamental PCIe switch ordering rules.

5.3 Transport-Agnostic Implementation Blueprint

Transitioning to a 0-kernel, transport-agnostic architecture requires precise coordination between the host CPU, the RDMA NIC, and the GPU memory controllers. The following Rust blueprint demonstrates how to properly orchestrate out-of-order EFA completions with strictly ordered PCIe transactions to safely unblock GPU execution without relying on an OS kernel context switch.

Rust

```
// This memory address is read continuously by the GPU's stall kernel.
```

```

let uvm_watcher_ptr = engine.alloc_uvm_watcher(old_val, new_val) {
    log::trace!("UVM State transitioned from {} to {}", old_val, new_val);
};

// Register the IMMCOUNTER expectation with the TransferEngine
engine.expect_imm_count(
    target_imm_id,
    expected_chunk_count,
    move || {
        // CALLBACK CONTEXT: Executed concurrently on the CPU callback thread
        // ONLY when the NIC Completion Queue has received exactly expected_chunk_count
        // immediate values matching `target_imm_id`.

        // At this exact microsecond, PCIe ordering guarantees that all payload
        // bytes are safely resident in the GPU's HBM.

        // Notify the GPU to unblock the CUDA stream
        // Write the READY status directly to the UVM flag via GDRCopy.
        // This sub-microsecond PCIe transaction bypasses the Linux kernel entirely.
        write_uvm_flag_via_gdrcopy(uvm_watcher_ptr, STATUS_READY);

        log::info!("KV Cache fully received. CUDA execution unblocked.");
    }
);

// Under the hood, the TransferEngine worker thread is executing a tight,
// lock-free polling loop against the `libfabric` CQ, translating out-of-order
// SRD completions into `IMMCOUNTER` increments.

```

6. Orthogonal Optimizations: Algorithmic Scheduling and Predictive Routing

While the core of this paper focuses strictly on the transport layer and communication libraries (e.g., mitigating ordered delivery constraints via [fabric-lib](#) and [aws-ofi-nccl](#)), resolving physical network bottlenecks is only half of the hyperscale equation. The logical distribution, scheduling, and predictive routing of the workload present orthogonal, yet equally critical, scaling challenges.

6.1 Context-Length Heterogeneity and Dynamic Meshes

At the 10,000+ GPU scale, handling extreme context-length heterogeneity exposes fatal flaws in static communication meshes. Traditional frameworks construct rigid, orthogonal domains for Data Parallelism (DP) and Context Parallelism (CP). Next-generation frameworks resolve this at the logical layer via advanced scheduling. For example, [ByteScale](#) [29] introduces Hybrid Data Parallelism (HDP) to dissolve the rigid DP/CP boundary in favor of a *dynamic mesh*. By employing data-aware sharding, sequences are routed to an optimal subset of devices, ensuring that short sequences bypass extensive ring-P2P communication.

6.2 Predictive MoE Routing and Expert Placement

Similarly, the data movement chaos inherent in sparse Mixture-of-Experts (MoE) routing can be heavily mitigated before the transport layer is invoked. Recent profiling of trillion-parameter models demonstrates that MoE expert selection is highly predictable both temporally and spatially [30]. By decentralizing popular experts and performing prefill-aware expert placement, operators can prevent severe network hot-spots from overwhelming specific RDMA NICs.

6.3 KV Cache Decoupling

Furthermore, as context windows scale exponentially, comprehensive taxonomies of the LLM data pipeline emphasize the absolute necessity of decoupling the logical structure of the KV cache from its physical storage [31]. This logical decoupling acts as the prerequisite enabler for the high-throughput, point-to-point disaggregated inference transfers discussed in Section 5.

6.4 Scope Limitation and Future Research Directions

While this paper provides a robust 0-kernel blueprint for mitigating physical network bottlenecks, the transition from transport-layer efficiency to total system goodput requires addressing the logical orchestration of data. As detailed in Section 6, optimizations such as [ByteScale's](#) dynamic meshes [29] and [predictive MoE placement](#) [30] act as essential multipliers. However, an exhaustive analysis of these algorithmic mechanics remains outside the scope of this manuscript.

To fully saturate the zero-copy, transport-agnostic data planes proposed herein, subsequent research should prioritize the following high-impact areas:

- **Context-Aware Mesh Scaling:** Future research should extend [ByteScale's](#) Hybrid Data Parallelism (HDP) [29] to perform real-time, context-aware re-meshing. By dynamically adjusting CP/DP group sizes based on live batch sequence lengths, systems can eliminate the internal fragmentation and "communication bubbles" that currently plague large-scale training.
- **Predictive Expert and KV-Cache Migration:** Building on the findings of [Patterns behind Chaos](#) [30], next-generation schedulers must integrate prefill-aware expert placement. By forecasting expert activation and [KV-cache](#) demand [31] before the decode phase begins, data can be migrated across the 0-kernel plane in the background, masking latency and preventing the NIC hot-spots typical of trillion-parameter models like DeepSeek-R1.
- **Logical-to-Physical Decoupling for Multi-Tenancy:** As highlighted in [A Survey of LLM × DATA](#) [31], the industry is moving toward a total decoupling of the logical KV cache from physical storage. Future work must investigate how transport-agnostic libraries like [fabric-lib](#) can natively support this decoupling to facilitate elastic scaling and dynamic preemptible instances without losing state.

By integrating these logical orchestration strategies with the 0-kernel physical architectures presented in this paper, AI infrastructure can finally breach the "Hyperscale Wall" and maintain peak efficiency regardless of hardware ordering constraints.

7. Conclusions and Strategic Directives

The data planes underpinning hyperscale AI must evolve from rigid, synchronized architectures to asynchronous, transport-agnostic frameworks. As clusters breach the 100,000-GPU threshold, relying on a single collective library fundamentally limits throughput, introduces cascading synchronization stalls, and enforces strict vendor lock-in.

To survive the "hyperscale wall" and the physical latencies of multi-tier Clos networks, operators must abandon GPU-driven memory copying in favor of host-driven, zero-copy abstractions (like **NCCLX**). Furthermore, migrating to hyperscale cloud fabrics (like AWS EFA) requires bridging ordered collective assumptions with out-of-order routing protocols (SRD) via essential shims like the [aws-ofi-nccl](#) plugin

[27]. Finally, the rise of disaggregated inference and MoE routing mandates a complete shift toward dynamic, asynchronous point-to-point streaming—standardizing on **NIXL 1.0.0 [28]** for production serving frameworks, or pure 0-kernel abstractions like **fabric-lib** for custom hyperscale engines.

To guide infrastructure deployments, operators must align software abstractions with physical network realities using the following prescribed framework:

Workload Scenario / Scale	Recommended Library	Architectural Characteristics / System Justification
Standard Synchronous Training (<10K GPUs)	Standard NCCL	Unmatched intra-node NVLink utilization and hardware-offloaded reduction math. Optimally designed for rigid, hardware-ordered fabrics but introduces straggler bottlenecks at hyperscale.
Cloud-Native Synchronous Training (AWS)	NCCL + aws-ofi-nccl plugin [27]	Bridges NCCL's in-order expectations to out-of-order transports (SRD) via Libfabric. Enables massive distributed training on instances like the p5, absorbing a slight CPU software-reordering tax to leverage EFA's bisection bandwidth.
Hyperscale Training (100K+ GPUs)	NCCLX (or custom zero-copy variants)	Host-driven, zero-copy architecture. Mitigates cross-datacenter physical latency across multi-tier Clos networks and eliminates GPU SM/HBM starvation by abandoning FIFO staging buffers in favor of direct RDMA transfers and Dynamic Queue Pair Load Balancing (DQPLB).
Production Disaggregated Inference & MoE	NIXL (v1.0.0+) [28]	The production standard for frameworks like vLLM and SGLang. Utilizes a native Libfabric backend to optimize SRD flow, providing zero-copy NVLink shared memory and high-throughput multi-rail NIC aggregation out-of-the-box.

Custom Hyperscale Inference / Sparse P2P	fabric-lib	Delivers the highest theoretical throughput for asynchronous P2P operations. Fully embraces unordered delivery via the 0-kernel IMMCounter abstraction, avoiding all software reordering jitter and proprietary silicon lock-in.
---	-------------------	--

8. References

1. Tianyuan Wu et al., "Adaptra: Straggler-Resilient Hybrid-Parallel Training with Pipeline Adaptation," *arXiv preprint arXiv:2504.19232v1*, Apr. 2025. [Online]. Available: <https://arxiv.org/abs/2504.19232>
2. Amazon Web Services, "Elastic Fabric Adapter (EFA) User Guide," AWS Documentation, 2026. [Online]. Available: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/efa.html>
3. A. Gangidi et al., "RDMA over Ethernet for distributed training at Meta scale," in *Proc. ACM SIGCOMM Conf.*, New York, NY, USA, 2024, pp. 57–70.
4. K. Qian et al., "HPN: A dual-plane architecture for large language model training," in *Proc. ACM SIGCOMM Conf.*, 2024.
5. N. Shazeer et al., "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. 5th Int. Conf. Learn. Represent. (ICLR)*, Toulon, France, Apr. 2017.
6. A. Singhvi et al., "Falcon: A reliable, low latency hardware transport," in *Proc. ACM SIGCOMM Conf.*, Coimbra, Portugal, Sep. 2025, pp. 248–263.
7. C. Zhao et al., "DeepEP: an efficient expert-parallel communication library," DeepSeek AI GitHub Repository, 2025.
8. Elastic Fabric Adapter for AI/ML and HPC workloads on Amazon EC2 - AWS Documentation, accessed May 6, 2026, [Online]. Available: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/efa.html>
9. Z. Jiang et al., "MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs," *arXiv preprint arXiv:2402.15627v1*, Feb. 2024. [Online]. Available: <https://arxiv.org/abs/2402.15627>
10. N. Licker et al., "fabric-lib: RDMA Point-to-Point Communication for LLM Systems," *arXiv preprint arXiv:2510.27656v2*, Apr. 2026. [Online]. Available: <https://arxiv.org/abs/2510.27656>
11. Meta, "torchcomms/ncclx," GitHub, 2026. [Online]. Available: <https://github.com/meta-pytorch/torchcomms/tree/main/comms/ncclx>
12. NVIDIA, "nccl," GitHub, 2025. [Online]. Available: <https://github.com/NVIDIA/nccl>
13. NVIDIA Corporation, "NVIDIA Dynamo adds support for AWS services to deliver cost-efficient inference at scale," NVIDIA Developer Blog, 2026.
14. NVIDIA Corporation, "Enhancing distributed inference performance with the NVIDIA Inference Transfer Library," NVIDIA Developer Blog, 2026.
15. P. Patel et al., "Splitwise: Efficient generative LLM inference using phase splitting," in *Proc. 51st ACM/IEEE Annu. Int. Symp. Comput. Archit. (ISCA)*, Buenos Aires, Argentina, Jun. 2024, pp. 118–132.
16. Perplexity AI, "Perplexity AI Releases TransferEngine and pplx garden to Run Trillion Parameter LLMs on Existing GPU Clusters," MarkTechPost, Nov. 2025.
17. Perplexity AI, "pplx-garden (fabric-lib)," GitHub, 2026. [Online]. Available: <https://github.com/perplexityai/pplx-garden>
18. W. Qiang et al., "Syncopate: Automatic Fine-Grained Compute-Communication Overlap via Chunk-Centric Scheduling," in *Proc. USENIX Symp. Oper. Syst. Design Implementation (OSDI)*, 2026.
19. T. Wu et al., "FALCON: Pinpointing and mitigating stragglers for large-scale hybrid-parallel training,"

arXiv preprint arXiv:2410.12588, 2024.

20. T. Wu et al., "[Attack of the Bubbles: Straggler-Resilient Pipeline Parallelism for Large Model Training](#)," in *Proc. USENIX Symp. Networked Syst. Design Implementation (NSDI)*, 2026.
21. R. Shi et al., "[Designing efficient small message transfer mechanism for inter-node MPI communication on InfiniBand GPU clusters](#)," in *Proc. 21st Int. Conf. High Perform. Comput. (HiPC)*, Goa, India, Dec. 2014, pp. 1–10.
22. P. Shamis et al., "[UCX: an open source framework for HPC HPC network APIs and beyond](#)," in *Proc. 2015 IEEE 23rd Annu. Symp. High-Perform. Interconnects*, 2015, pp. 40–43. Source code available: <https://github.com/openucx/ucx>
23. L. Shalev et al., "[A cloud-optimized transport protocol for elastic and scalable HPC](#)," *IEEE Micro*, vol. 40, no. 6, pp. 67–73, Nov. 2020.
24. M. Si et al., "Collective Communication for 100k+ GPUs," *arXiv preprint arXiv:2510.20171v4*, Jan. 2026. [Online]. Available: <https://arxiv.org/abs/2510.20171>
25. Y. Zhong et al., "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implementation (OSDI)*, Santa Clara, CA, USA, Jul. 2024, pp. 193–210. [Online]. Available: <https://arxiv.org/pdf/2401.09670>. Source code available: <https://github.com/LLMServe/DistServe>
26. Y. Zhu et al., "[Congestion Control for Large-scale RDMA Deployments](#)," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 4, pp. 523–536, Aug. 2015.
27. Amazon Web Services, "aws-ofi-nccl: A plugin which lets EC2 developers use libfabric as network provider while running NCCL applications," GitHub, 2026. [Online]. Available: <https://github.com/aws/aws-ofi-nccl>
28. Amazon Web Services, "Get started with EFA and NIXL for inference workloads on Amazon EC2," Amazon Elastic Compute Cloud User Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/efa-start-nixl.html>
29. H. Ge et al., "ByteScale: Communication-Efficient Scaling of LLM Training with a 2048K Context Length on 16384 GPUs," in *Proc. ACM SIGCOMM 2025 Conference*, Coimbra, Portugal, Sep. 2025, pp. 963–978.
30. Y. Yu et al., "[Patterns behind Chaos: Forecasting Data Movement for Efficient Large-Scale MoE LLM Inference](#)," in *Proc. ISCA*, 2026.
31. X. Zhou et al., "[A Survey of LLM x DATA](#)," *arXiv preprint arXiv:2505.18458*, May 2025.